

JurnaPod date-/time-/timezone handling — Current state and simplification proposals

Current API state

The **JurnaPod** repository has evolved a fairly elaborate standard for date/time handling, rooted in a desire to avoid past inconsistencies. The policies, code and ADRs describe and enforce a canonical API contained in `packages/shared/src/schemas/datetime.ts`. Key features of the current implementation include:

Canonical types and conversions

- **Canonical internal format** – a UTC ISO `YYYY-MM-DDTHH:mm:ssZ` string. `UtcIsoSchema` validates these strings and rejects offsets ¹.
- **Date-only type** – `DateOnlySchema` for `YYYY-MM-DD` business-local dates ¹.
- **Timezone schema** – IANA names validated by `TimezoneSchema`, plus helpers such as `isValidTimeZone` and `resolveBusinessTimezone` (choose outlet timezone → company timezone → error) ¹.
- **Namespaced helpers** – `toUtcIso` and `fromUtcIso` convert between various representations (Date objects, epoch milliseconds, business dates, date ranges) and the canonical UTC ISO string. These helpers also handle half-open intervals and as-of semantics ².
- `nowUTC()` – returns the current instant as a UTC ISO string; used instead of `new Date()`.

Policies and conventions

- **Suffix conventions** – the policy defines `*_ts` for epoch-millisecond timestamps, `*_at` for ISO strings, `*_date` for business dates, and `created_at` / `updated_at` for audit timestamps ³.
- **Time-zone resolution** – functions must resolve time zones using outlet → company → error and never fall back to UTC; all IANA names must be valid ⁴ `【869884557143811±L4-L17】`.
- **Half-open date ranges** – queries use inclusive `start_date` and exclusive `end_date` boundaries.
- **Prohibited patterns** – the docs prohibit using raw `Date`, `new Date()`, `toISOString().slice(0, 10)`, offset ISO strings, manual string concatenation or slicing, or fallback to UTC ⁴ `【869884557143811±L4-L17】`.
- **Migration plan** – the ADR and consolidation plan describe renaming old helpers to namespaced functions and replacing ad-hoc date handling.

Real-world usage and inconsistencies

Despite the comprehensive standard, the code base contains many exceptions:

- **Reimplementation of** `DateOnlySchema` – numerous schema files (e.g., `cash-bank.ts`, `fiscal-years.ts`) define their own regex-based `DateOnlySchema` and validate `created_at` /

`updated_at` with `z.string().datetime()` rather than the canonical `UtcIsoSchema`. This leads to inconsistent validation ⁵.

- **Manual date slicing** – many modules, especially front-end components and some API helpers, derive date strings by calling `new Date().toISOString().slice(0, 10)` or slicing an ISO string. For example, the back-office export utilities build filenames using `new Date().toISOString().slice(0, 10)` ⁶, and the `common-utils.ts` file defines `formatDateOnly` and `formatDateOnlyFromUnknown` by slicing strings or manually constructing dates ⁷.
- **Mixed use of ISO strings and epoch times** – some modules store business events as ISO strings while others use epoch milliseconds; this increases cognitive load.
- **Complex timezone resolution** – the dual-mode resolution (outlet → company → error) introduces many branches. Some modules integrate this logic, while others ignore it and default to `UTC` or system time, creating subtle bugs.

This fragmentation means developers must learn multiple patterns and constantly cross-check the policy documents, which defeats the goal of simplicity.

Simplification proposals

The user has asked to "ignore the current standardization" and make date/time handling simpler. Below are high-level recommendations that would simplify both the API surface and implementation while preserving correctness:

1. Adopt a single internal representation: epoch milliseconds

Using a single numeric representation for all instants (e.g., `number` representing milliseconds since the Unix epoch) eliminates confusion about ISO strings versus dates versus epoch times. Epoch milliseconds are timezone-agnostic and monotonic, and they can be readily converted to JavaScript `Date`, SQL timestamps or ISO strings when needed. For example:

```
// canonical internal representation
export type TimestampMs = number;
```

Whenever the API accepts or returns date/time information, use epoch milliseconds. For display or storage in databases that require date strings, convert at the boundary (e.g., in controllers or ORM). Conversions can be simple helpers:

```
function toIsoString(ts: TimestampMs): string {
  return new Date(ts).toISOString();
}
function fromIsoString(iso: string): TimestampMs {
  return Date.parse(iso);
}
function toDateOnlyString(ts: TimestampMs, tz?: string): string {
  // If a timezone is provided, convert via Intl or a lightweight library
```

```
return new Date(ts).toLocaleDateString("en-CA", { timeZone: tz });
}
```

A single representation reduces the number of suffixes developers must remember and eliminates the need for multiple overlapping schemas.

2. Simplify timezone handling

The current approach attempts to resolve a timezone from multiple sources (outlet → company → error), forbids fallback to UTC and requires explicit IANA names. This adds complexity and leads to duplication across modules. A simpler approach is:

- **Single default timezone per company** – store a company-wide timezone configuration. If different outlets need different time zones, store it on each record (e.g., `outlet.timeZone`), and always pass the timezone explicitly when converting a timestamp. Do not attempt to resolve automatically; require the caller to specify the timezone or rely on a default.
- **Use `Intl.DateTimeFormat` or a lightweight library** – avoid implementing timezone conversions yourself. Modern JavaScript provides `Intl.DateTimeFormat` with `timeZone` option. For more complex operations (e.g., differences across DST changes), use a library like [Luxon](#) or the forthcoming `Temporal` API rather than duplicating logic. This ensures correctness and reduces maintenance.
- **Drop dual-mode fallback** – when a timezone is not provided, either assume UTC or throw an error. Fallback to company or outlet adds uncertainty; requiring explicit input is simpler.

3. Standardize date-only handling

A `YYYY-MM-DD` string is intuitive and often used for date pickers and filters. Keep one `DateOnly` type and helper to convert epoch milliseconds to date-only strings in a given timezone (defaults to UTC). Avoid building date strings by slicing ISO strings; instead use `Intl.DateTimeFormat`:

```
function toDateOnly(ts: TimestampMs, tz = "UTC"): string {
  return new Intl.DateTimeFormat("en-CA", { timeZone: tz }).format(new
    Date(ts));
}
```

When parsing a date-only string into an instant, assume the date at midnight in the chosen timezone:

```
function fromDateOnly(date: string, tz = "UTC"): TimestampMs {
  // Use Date.parse on ISO with timezone suffix
  // For example "2024-05-01T00:00:00" in the given timezone
  return Date.parse(`${date}T00:00:00${offsetForTimezone(tz)}`);
}
```

Using a small set of helpers for date-only conversions prevents repetition and eliminates regex-based `DateOnlySchema` duplication.

4. Eliminate redundant suffixes and naming conventions

The current suffix taxonomy (`*_ts`, `*_at`, `*_date`) is well intentioned but unnecessary when a single internal representation is used. A simplified approach could be:

- Use `created_at` / `updated_at` for audit timestamps, stored as epoch ms.
- Use descriptive field names for business events, e.g., `invoice_date` (date-only string), `payment_time` (epoch ms), without requiring suffixes. Document the type for each field rather than inferring from the suffix.

This reduces cognitive load and allows developers to name fields according to domain meaning.

5. Remove manual string manipulation

Throughout the code base, there are patterns like `new Date().toISOString().slice(0, 10)` or `value.slice(0, 10)`. These are fragile and error-prone, especially around daylight-saving transitions. Replace them with helper functions based on `Intl.DateTimeFormat` or a date library. For example, the back-office export file names can use:

```
const today = toDateOnly(Date.now(), /* timezone */);
const filename = `items-filtered-${today.replace(/-/g, "")}.csv`;
```

Similarly, forms and filters should convert dates using these helpers rather than slicing strings.

6. Provide clear guidelines rather than rigid enforcement

The existing policy reads like an RFC, listing prohibited patterns and enforcement guidelines. While consistency is important, heavy enforcement discourages developers from writing simple code. A simpler guideline document could be:

1. Store all timestamps as epoch milliseconds.
2. Use `Intl.DateTimeFormat` or a proven library for formatting and parsing dates in specific time zones.
3. For date-only filters, use `YYYY-MM-DD` strings derived via helpers.
4. Prefer explicit timezone parameters rather than implicit resolution.

A short set of rules is easier to follow and can be enforced by lint rules or test helpers.

Summary

JurnaPod's current date/time standardization provides strong guarantees but introduces complexity: multiple suffix-based types, dual-mode timezone resolution, namespaced conversion functions and numerous prohibited patterns. Code across the repository still re-implements `DateOnly` schemas and slices strings manually ⁵ ⁶. To simplify the API, the project can:

- **Converge on a single internal representation**, such as epoch milliseconds, for all timestamps.

- **Use existing libraries and the `Intl` API** for timezone conversions rather than custom resolution logic.
- **Provide straightforward helpers** for converting epoch ms to ISO strings and date-only strings and vice versa.
- **Reduce the naming taxonomy** to avoid suffix overload and encourage descriptive field names.
- **Avoid manual string slicing**; rely on robust formatting functions instead ⁷.
- **Document a short set of guidelines** instead of a long list of rules and anti-patterns.

These changes would reduce the learning curve for developers, minimise duplicated code, and still allow correct handling of dates, times and time zones in the JurnaPod API.

¹ ² `datetime.ts`

<https://github.com/s18id/jurnapod/blob/9f45e1995c52bc5be50688787b5e197c29176c2d/packages/shared/src/schemas/datetime.ts>

³ ⁴ `date-time-standardization.md`

<https://github.com/s18id/jurnapod/blob/9f45e1995c52bc5be50688787b5e197c29176c2d/docs/policies/date-time-standardization.md>

⁵ `cash-bank.ts`

<https://github.com/s18id/jurnapod/blob/9f45e1995c52bc5be50688787b5e197c29176c2d/packages/shared/src/schemas/cash-bank.ts>

⁶ `items-prices-export-utils.ts`

<https://github.com/s18id/jurnapod/blob/9f45e1995c52bc5be50688787b5e197c29176c2d/apps/backoffice/src/features/items-prices-export-utils.ts>

⁷ `common-utils.ts`

<https://github.com/s18id/jurnapod/blob/9f45e1995c52bc5be50688787b5e197c29176c2d/apps/api/src/lib/shared/common-utils.ts>